Performance Study of Register-Width for K-Means Clustering: Scalar C versus XMM and YMM SIMD Implementations

Project Proposal | Advanced Computer Architecture (CEPARCO), 1st Term AY 2026-2027 Group 2, Section S01

Daniel Jedrick C. Dabuit, Ethan Daniel B. Magbatoc, Jose Bryan L. Perez, Tara Ysabel P. Uy

Revisions from the original submission are highlighted in yellow.

0. Abstract

This project investigates how SIMD register-width optimization techniques affect the performance of K-means clustering on an x86-64 CPU. The study conducts a comparison of a scalar C as the baseline, SSE XMM registers (128-bit, four single-precision values per instruction), and AVX2 YMM registers (256-bit, eight values per instruction). The implementations will use the same datasets, centroids, and empty-cluster handling for comparison. Experiments will vary the data points, number of clusters, and the optimization techniques to investigate how workload size and computation demand affects performance. The results will show the benefits and limitations of wider SIMD registers and loop optimizations for K-means clustering on the CPU.

1. Description of the Proposed Project

K-means clustering partitions N points into K clusters. Starting from K initial centroids, it repeats two steps until the centroids stop moving or a maximum number of iterations is reached: (1) the assignment step labels each point with its nearest centroid, and (2) the update step moves each centroid to the mean of the points assigned to it. The distance used is the squared Euclidean distance, $d^2 = (x - cx)^2 + (y - cy)^2$. The square root is omitted because it does not change which centroid is nearest.

1.a Inputs

Input	Description
X[], Y[]	Single-precision (float32) coordinates of the N points, stored as two separate arrays (structure-of-arrays)
N	Number of data points

K	Number of clusters
MAX_ITER	Maximum number of iterations
TOL	Convergence threshold on centroid movement
SEED	Fixed random seed, so all versions receive identical data and identical initial centroids
EMPTY_POLICY	How an empty cluster is handled: A (keep the previous centroid) or B (relocate). See Section 1.d.
FIXED_ITERS	Number of iterations run in timing mode (with TOL = 0), so every version does identical work

The datasets are synthetic Gaussian blobs generated with a fixed seed. Planned sizes are N = 2^16, 2^20, and 2^24 with K = 4, 8, 16, and 32, plus boundary sizes (N = 7, 1003, 1005) to test the handling of leftover points. The initial centroids are the first K points of the array. The program will validate that the coordinate arrays have equal lengths, N and K satisfy the required constraints, and that all coordinates are finite numerical values. All implementations will have the same input data, initial centroids, and parameters to ensure a fair comparison.

1.b Proposed Process

i. Processes to be parallelized

The **assignment step** is the main target. For each point, the squared distance to each centroid is computed and the smallest is kept. A point's result does not depend on any other point, so the step is data-parallel. With XMM registers, four points are processed per instruction; with YMM registers, eight. This step performs N × K distance computations per iteration and accounts for most of the work. Hamerly reports that the innermost loop over the centers is where Lloyd's algorithm spends the majority of its time [6], and scikit-learn documents the average cost as O(k n T) for n samples and T iterations [8]. The same kernel design is built at each register width, so that width is the only change between versions.

The **centroid accumulation** (per-cluster sums of x and y, and member counts) is a secondary target, to be vectorized only if time permits.

ii. Related works and how this project differs

Lloyd's algorithm. Lloyd's algorithm [1] is the standard formulation of k-means and is the algorithm implemented in this project, unchanged.

Algorithmic accelerations. Hamerly [6] accelerates Lloyd's algorithm while giving the same answer. It uses distance bounds and the triangle inequality to avoid distance computations; in the author's single-threaded experiments it skips the innermost loop over the centers 80% of the time or more, and it is usually the fastest of the compared algorithms on the 2-dimensional datasets. scikit-learn [8] uses Lloyd's algorithm by default and offers an Elkan variant that also uses the triangle inequality, at the cost of an extra n_samples × n_clusters array. These methods reduce how many distances are computed. This project keeps full Lloyd's algorithm, so every version computes the same N × K distances and only the cost of each distance changes with the register width. Combining both approaches is possible future work.

SIMD implementations. Al Hasib et al. [7] is the closest existing SIMD implementation. It combines a block data layout, an in-register argmin, SIMD compression, and OpenMP threads on integer data, and evaluates 128-bit SSE and 512-bit AVX-512 on a Haswell CPU and a Xeon Phi KNL. It reports a 4.5x performance improvement for the compressed SSE version on Haswell with a single thread. It also reports that compiler auto-vectorization reached only 1.4x (SSE) and 8.2x (AVX-512), against ideal speedups of 4 and 16, and executed about twice as many instructions as the hand-written SSE code. The paper states that the 256-bit (AVX2) results are not shown and can be extrapolated. This project differs in four ways: it uses float32 data and a single thread, it keeps one fixed data layout, it measures the 256-bit YMM case directly next to the 128-bit XMM case, and it tests whether handwritten assembly beats the compiler's auto-vectorized C on our CPU and compiler.

GPU implementations. FAISS [2] is a library for similarity search and clustering, and its paper notes that GPUs are well suited to data-parallel tasks such as distance computation. NVIDIA's cuVS library [3] provides a GPU-accelerated k-means with the same two steps, and its guide notes that the expensive part is repeatedly comparing many rows against many centroids. Reference [7] also describes earlier GPU work that computes the distances on the GPU and updates the centroids on the CPU. These depend on GPU hardware, while this project targets CPU SIMD registers; a CUDA (SIMT) version is only an optional extension.

How this project differs. K-means already has SIMD [7] and GPU [2], [3] implementations. Among the sources we reviewed, none measures 128-bit and 256-bit register width on exact Lloyd's k-means as a controlled comparison; the closest work leaves the 256-bit case to extrapolation. This project therefore does not claim a new algorithm. Its contribution is a controlled measurement of how register width affects the assignment step, with a defined empty-cluster policy, to show where the speedup comes from and what limits it, such as memory bandwidth and the steps that remain sequential.

iii. Plan for parallelizing the process

The algorithm is transformed to expose SIMD parallelism as follows:

1. **Data layout.** Store X[] and Y[] as separate arrays so that one load fills a register with consecutive x (or y) values.
2. **Remove the square root.** Compare squared distances only. Hamerly also notes that square roots can be avoided when only the closest center is needed [6].

3. **Broadcast.** For each centroid, copy cx and cy into every slot of a register. This follows the centroid-oriented approach described in [7], where a group of points is compared with the same centroid at a time.
4. **Distance.** Compute $dx = X - cx$, $dy = Y - cy$, and $d^2 = dx \cdot dx + dy \cdot dy$ for all points in the register at once, using separate multiply and add instructions (no fused multiply-add) so results match the C version exactly.
5. **Branch-free minimum.** Keep, per slot, the smallest d^2 so far and the index of the centroid that produced it, using a compare and a blend instead of a conditional jump. Ties go to the lower centroid index in all versions. The kernel also stores each point's smallest d^2, which is needed for the inertia and for empty-cluster relocation (see Empty-cluster handling below).
6. **Centroid loop.** Iterate over the K centroids sequentially, then store the labels of the whole group of points.
7. **Leftover points.** When N is not a multiple of 4 (XMM) or 8 (YMM), finish the remaining points with a scalar loop. For N = 1003, the YMM kernel performs 125 steps of 8 points and the scalar loop handles the last 3.
8. **Width variants.** The XMM kernel (4 points per step) and the YMM kernel (8 points per step) share one design; only the register width, loop stride, and leftover size change. Both use the AVX (VEX) instruction encoding, so the 128-bit kernel is AVX-128 and the 256-bit kernel is AVX-256.

The centroid update is harder to vectorize because many points add into the same cluster total. One option is to process one cluster at a time: compare the labels against the cluster number, mask out other points, and sum the rest. This requires K passes and is only worthwhile for small K.

Four code versions will be built: scalar C (compiler vectorizer off), auto-vectorized C, the XMM kernel, and the YMM kernel. A CUDA (SIMT) version is an optional extension if time allows after the CUDA lecture.

Empty-cluster handling. A cluster is empty when no point is assigned to it, which leaves its mean undefined (a division by zero). Nielsen and Nock show that Lloyd's algorithm can produce such empty-cluster events, and report that they become more frequent as K or the dimension grows, and with random seeding more than with k-means++ seeding [9]. Because our initial centroids are the first K points, duplicate early points can also cause them. Every version applies the same rule so that results stay comparable. As in [9], ties between equally close centroids go to the lowest index. Two policies are implemented and compared:

- **Policy A (default), keep:** the empty cluster's centroid stays where it was, and the event is counted.
- **Policy B, relocate:** the empty centroid moves to the point with the largest squared distance to its assigned centroid (lowest index on ties). If several clusters are empty, the next farthest points are used in order. Reference [9] re-seeds empty clusters with k-means++ or global k-means; this deterministic rule is used instead so that all versions stay identical.

The assignment kernel already computes each point's smallest d^2, so it stores it, and Policy B needs only a scan for the maximum, with no extra distance work. Detection is a check of the K member counts after each update step. The scan runs only when an empty cluster exists, so it is rare and stays sequential. **Verification:** two test datasets force empty clusters. The first is the five-point example of [9] (n = 5, K =

3), with the initial centroids set to the points (0.03, 0.92), (0.66, 0.79), and (0.6, 0.85); the second assignment then leaves the second cluster empty. The second is a duplicatepoint case: N = 8 with the first three points identical and K = 3, so every point ties to centroid 0 and clusters 1 and 2 are empty. All versions must produce identical labels, counts, and centroids under both policies. We report the events per run, the extra time of Policy B, and its effect on the inertia.

iv. Processes that will not be parallelized

- Data generation and input validation
- Initial centroid selection
- The iteration loop and the convergence check (K comparisons per iteration)
- The division of the sums by the counts to obtain the new centroids (K divisions)
- Empty-cluster detection and relocation (rare, so it stays sequential; counted and timed separately)
- Timing, accuracy checking, and reporting
- The centroid update, unless its vectorized version is completed

Because these parts remain sequential, the overall speedup is expected to be lower than that of the assignment step alone (Amdahl's law). Each stage is therefore timed separately.

1.c Outputs

- The final label of every point, the number of members in each cluster, and the final centroids
- Execution time per stage for each version: initial centroid selection, assignment with member counting, centroid recomputation, and full runtime
- Speedup of the XMM and YMM versions over C, summarized by the geometric mean across the tested sizes
- **Similarity score:** the percentage of points assigned the same label as in the C version (expected 100%), together with the largest centroid difference and the inertia (sum of squared distances to the centroids, as reported by cuVS [3] and scikit-learn [8]) of each version
- Number of empty-cluster events per run, and the extra time that handling them costs
- Efficiency (speedup divided by the ideal 4 or 8), points processed per second, and the measured memory bandwidth

Timing uses QueryPerformanceCounter, with one warm-up run followed by 30 timed runs averaged, in Release (primary) and Debug (secondary) configurations.

Controlled benchmark design. To keep the comparison fair, only one factor changes at a time and everything else is fixed:

- **Identical work:** same data, seed, and starting centroids. Timing runs use a fixed iteration count (FIXED_ITERS, TOL = 0), so every version does the same work. Convergence runs are used only for correctness.
- **Timed region:** only the clustering is timed, per iteration and in total, excluding data loading and setup, as in [6]. The assignment kernel is also timed on its own. No input/output or printing inside the timer.

- **Memory:** 32-byte aligned arrays, touched once before timing so that page faults are not measured.
- **Repetitions:** one warm-up, then 30 timed runs. Report the mean, median, standard deviation, and minimum. Page 4
- **System:** one thread pinned to one core (a single core is used in [6]); laptop plugged in, high-performance power plan, background applications closed. The Turbo Boost state is recorded and, if the system allows, fixed, as in [7]. CPU model, cache sizes, compiler version, and flags are recorded.
- **Build:** Release (/O2). Scalar C is built with the vectorizer disabled (#pragma loop(no_vector)). A second C build lets the compiler auto-vectorize (/arch:AVX2); [7] found that auto-vectorized code stayed well below the ideal speedup, and we test whether this holds for our compiler and CPU. Disassembly is checked to confirm that no fused multiplyadd is emitted and that the expected vector width is used.
- **Sweeps:** N and K follow Section 1.a (N = 2^16, 2^20, 2^24; K = 4, 8, 16, 32), with additional smaller sizes of N (down to 2^10) so that the data also fit in the lower cache levels. One factor changes at a time.
- **Memory ceiling:** a simple copy test measures the machine's practical memory bandwidth, so that measured throughput can be compared with it to tell memory-limited runs from compute-limited ones.
- **Validity:** a run counts only if its labels match the C version exactly. Mismatches are reported, never averaged in.

2. Proposed Target Implementation Platform

An x86-64 CPU supporting SSE and AVX2, running Windows. The driver, baseline, data generation, correctness check, and timing are written in C and built in Visual Studio. The SIMD kernels are written in x86-64 assembly (NASM) and linked with the C program. Instruction details are checked against the Intel Intrinsics Guide [4] and the Intel 64 and IA-32 Architectures Software Developer's Manual [5]. The test machine (CPU, cache sizes, RAM, OS, and Visual Studio version) is documented in the report.

3. Mid-Project Milestones

Milestone #2 (Week 10): mid-project update demo

- Data generator and timing code completed (fixed-iteration mode, aligned memory, pinned core)
- Sequential C k-means completed (vectorizer off) and verified on a small hand-worked example
- Empty-cluster handling (Policies A and B) implemented in C and verified on the two forcedempty test datasets
- Memory-bandwidth copy test completed
- XMM assignment kernel working and producing the same labels as the C version
- YMM kernel started, with preliminary C versus XMM timing results

Milestone #3 (Week 13): complete submission and demo

- YMM assignment kernel completed and verified

- Auto-vectorized C build compared against the hand-written kernels
- Boundary sizes tested
- Full benchmarks across all N and K under the controlled protocol (Release; Debug secondary)
- Empty-cluster results: event counts, cost of Policy B, and effect on the inertia
- Optional vectorized centroid update and optional CUDA version
- GitHub repository with source code, discussion of the parallel algorithms, and the sequential versus parallel execution time comparison
- Optional conference paper

4. References

1. S. P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Information Theory*, vol. 28, no. 2, pp. 129-137, 1982. https://www.stat.cmu.edu/~rnugent/PCMI2016/papers/LloydKMeans.pdf
2. J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535-547, 2021. https://arxiv.org/abs/1702.08734
3. NVIDIA cuVS, “K-Means” clustering guide. https://docs.nvidia.com/cuvs/user-guide/api-guides/clustering-guide/k-means
4. Intel, *Intel Intrinsics Guide*. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html
5. Intel, Intel 64 and IA-32 Architectures Software Developer’s Manual. https://www.intel.com/content/www/us/en/content-details/825743/intel-64-and-ia-32-architectures-software-developer-s-manual-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4.html
6. G. Hamerly, “Making k-means even faster,” Proc. 2010 SIAM International Conference on Data Mining (SDM), pp. 130-140, 2010. https://cs.baylor.edu/~hamerly/papers/sdm_2010.pdf
7. A. Al Hasib, J. M. Cebrian, and L. Natvig, “A vectorized k-means algorithm for compressed datasets: design and experimental analysis,” J. Supercomputing, vol. 74, pp. 2705-2728, 2018. https://doi.org/10.1007/s11227-018-2310-0
8. scikit-learn developers, “sklearn.cluster.KMeans” documentation. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
9. F. Nielsen and R. Nock, "Further heuristics for k-means: The merge-and-split heuristic and the (k,l)-means," arXiv:1406.6314, 2014. https://arxiv.org/pdf/1406.6314